

Byte Streams

Handling Raw Data & Media Files

codehive

codehive

What are Byte Streams?

In Java, **Byte Streams** are used to perform input and output of 8-bit bytes. Unlike Character Streams (which handle text), Byte Streams are designed for "raw" binary data, such as images, audio, video, or executable files.

The Base Classes:

- `InputStream`: The abstract base class for all byte input streams.
- `OutputStream`: The abstract base class for all byte output streams.

codehive

Copying a Binary File

This implementation uses `FileInputStream` and `FileOutputStream` to copy an image byte-by-byte. We utilize the **Try-with-Resources** syntax to manage multiple streams simultaneously.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class CopyFile {
    public static void main(String[] args) {
        // Multiple resources separated by semicolons
        try (FileInputStream input = new FileInputStream("image.jpg");
            FileOutputStream output = new FileOutputStream("copy.jpg")) {

            int i;
            while ((i = input.read()) != -1) {
                output.write(i); // Writing the raw byte
            }
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("Error handling file.");
        }
    }
}
```

Key Takeaways

1. **-1 Check:** The `read()` method returns the byte as an `int`. It returns **-1** when the end of the file is reached.
2. **Media Files:** Character streams (`FileReader`) should *never* be used for images or PDFs as they might modify the bytes based on character encoding.
3. **Resource Order:** In Try-with-Resources, resources are closed in the **reverse order** in which they were created.

Binary Data Mastery with

codehive

Handling Bytes with Precision.